Milestone 3 Documentation: Airflow Orchestration & Monitoring

1. Introduction

This project implements an ETL (Extract, Transform, Load) pipeline using Apache Airflow. The goal is to automate data processing tasks, monitor execution, and integrate email notifications for failure alerts.

2. Objectives

- Automate ETL workflow using Airflow DAGs

- Schedule tasks (daily/hourly execution)

- Implement logging for monitoring

- Integrate email notifications for task failures

- Store intermediate and final processed data

3. Tools & Technologies Used

- Apache Airflow (Docker setup)

- Python (Pandas, smtplib)

- Gmail SMTP (for email alerts)

- Docker & Docker Compose

4. Project Architecture

Extract → Transform → Load

5. DAG Configuration

DAG ID: etl_pipeline_final_email_alert

Schedule: @daily

Start Date: January 1, 2024

Retries: 1

6. ETL Pipeline Implementation

6.1 Extract Phase

Reads dataset (employee.csv), selects rows, saves as data.csv

6.2 Transform Phase

Reads data.csv, removes null values, modifies column, saves as transformed_data.csv

6.3 Load Phase

Reads transformed_data.csv and logs output

7. Logging Implementation

Logging is used to track execution flow and debug errors.

8. Scheduling

DAG runs daily using @daily (can also use @hourly or cron expressions)

9. Email Notification Integration

Steps:

1. Enabled Gmail 2-Step Verification

2. Generated App Password

3. Configured SMTP in .env file

4. Restarted Docker containers

5. Implemented email function using smtplib

6. Attached failure callback to tasks

7. Tested using intentional failure

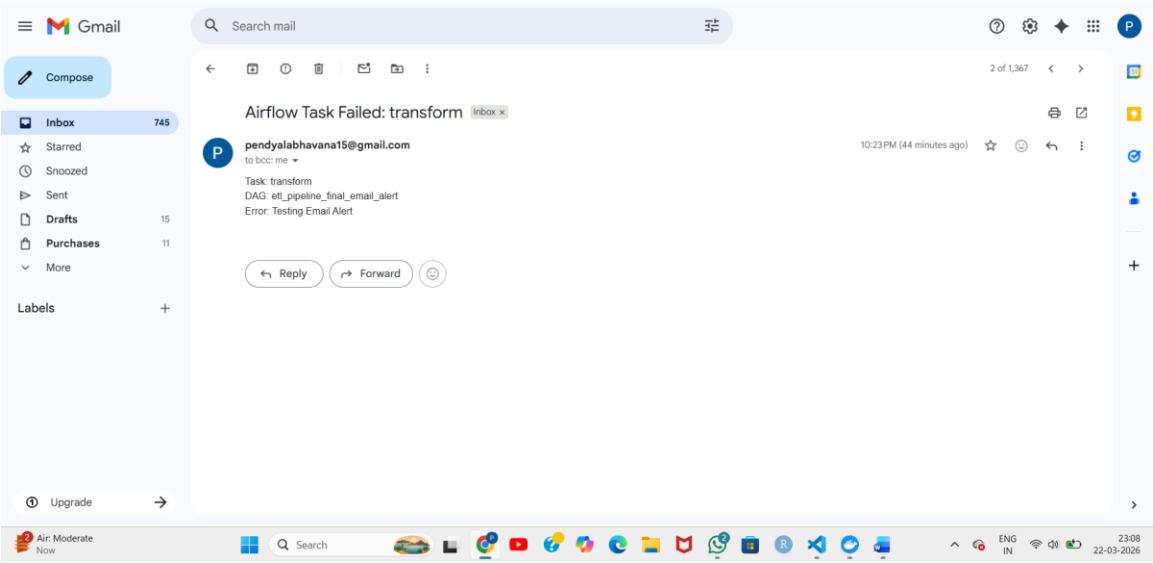
10. Final Output

- Automated ETL pipeline

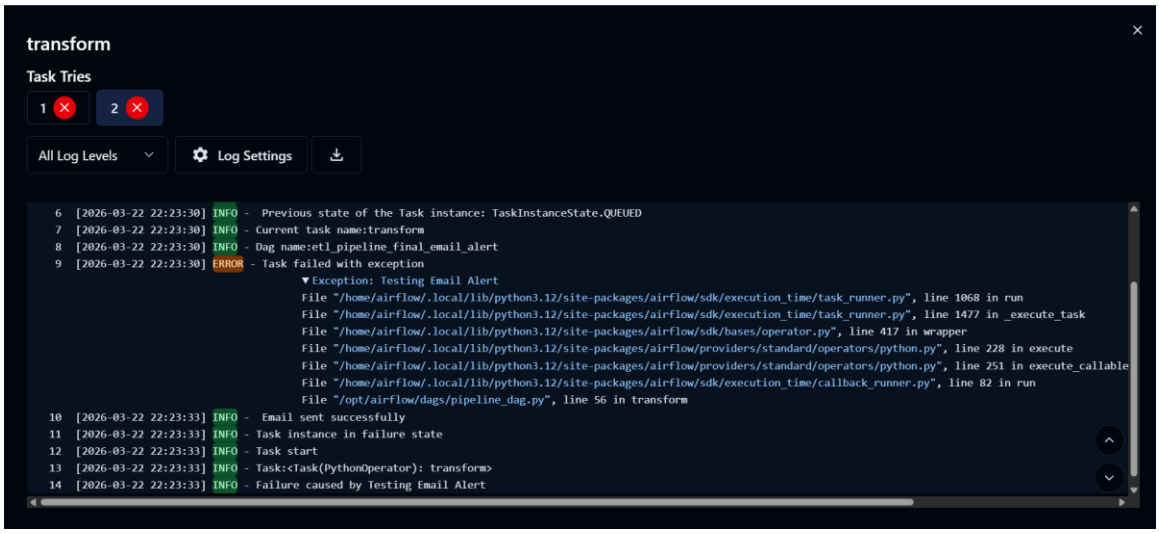
- Data stored at each stage

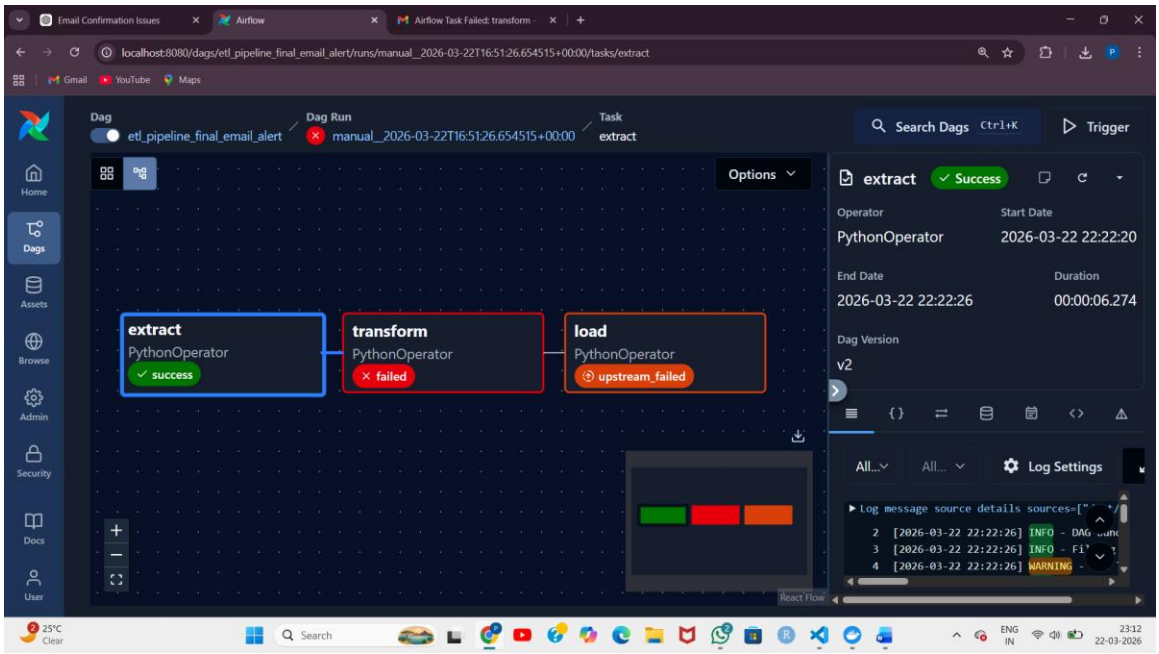
- Logs showing execution

- Email alerts on failure



When any task fails, Airflow automatically sends an email with task details and error message.





11. Conclusion

The project successfully demonstrates workflow orchestration, data processing, monitoring, and alerting using Apache Airflow.